

Práctica (1): PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Santiago Alexander Bravo Rodriguez - 2224734

Christian Felipe Barbosa Pinto - 2212911

https://github.com/SU_REPOSITORIO

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander

1 de septiembre de 2026

Abstract—Abstract— This laboratory practice focused on the design, implementation, and evaluation of user-defined signal processing blocks within the GNU Radio Companion environment. The primary goal was to move beyond the use of predefined libraries by developing three synchronous Python-based components: an accumulator, a discrete differentiator, and a statistical analyzer. Test signals were introduced to assess and validate the logical and mathematical behavior of each block. During the implementation process, several issues in the original code were detected and corrected, including a discontinuity problem in the accumulator caused by the loss of state between consecutive sample batches. The experimental results demonstrated that the modified algorithms closely matched their theoretical behavior, confirming their reliability. Overall, the practice highlighted the flexibility of Software Defined Radio (SDR) for developing customized signal-processing solutions, particularly for metric analysis and noise reduction in real-world communication systems.

Index Terms—Gato de ojos verdes. (Cambiar por las que usen)

I Introducción

El avance continuo de la tecnología en el área de las telecomunicaciones ha favorecido la transición hacia la Radio Definida por Software (SDR, por sus siglas en inglés), permitiendo trasladar gran parte del procesamiento de las señales a plataformas computacionales. En esta práctica se exploró el uso de GNU Radio Companion (GRC) como herramienta para el procesamiento de señales en tiempo real. El trabajo estuvo orientado no solo al manejo de la interfaz gráfica, sino también al desarrollo e implementación de algoritmos personalizados directamente mediante código. El propósito principal fue fortalecer las habilidades necesarias para programar y manipular funciones de procesamiento de señales utilizando Python. Asimismo, la creación de bloques personalizados permitió profundizar en el tratamiento de señales de tipo stream y mejorar la capacidad para

analizar diferentes fenómenos asociados con los canales de comunicación.[1].

II Metodología

El desarrollo del laboratorio se estructuró en dos fases complementarias: la gestión del entorno colaborativo y la codificación algorítmica. Para la primera fase, se empleó el sistema de control de versiones Git, mediante el cual se crearon ramas de desarrollo individuales a partir de una rama principal. Todo el código se administró de manera local y se sincronizó en la nube para consolidar el trabajo grupal. Para la segunda fase, la manipulación de las señales requirió la creación de bloques síncronos implementados en el lenguaje de programación Python. Dentro del método `work()`, se extrajeron los vectores de datos utilizando los arreglos unidimensionales `input_items` y `output_items`. Se programaron tres bloques principales: un acumulador, un diferenciador y un tercer bloque que resumía características estadísticas de la señal, tales como el promedio, la media cuadrática, el valor RMS, la potencia promedio y la desviación estándar. A partir de esta estructura, se ensamblaron los diagramas de bloques en la interfaz gráfica y se ejecutaron las simulaciones correspondientes. Las figuras obtenidas a partir de estas simulaciones permitieron comprobar que los valores estadísticos coincidían con la base teórica matemática de las señales inyectadas.

Acumulador

Para comprobar el funcionamiento del bloque acumulador, se implementó el montaje mostrado en la Figura 1. Como señal de entrada se utilizó una onda cuadrada acondicionada previamente, con valores que variaban de forma simétrica entre -1 y 1 .

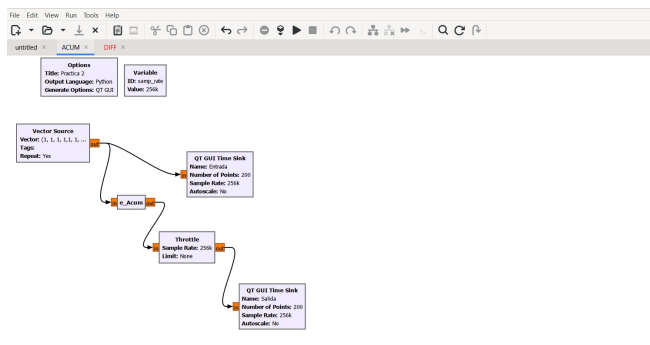


Figura 1: Diagrama de bloques del acumulador.

En la simulación del bloque acumulador se evidenció que el código inicial reiniciaba el valor de la suma cada vez que GNU Radio procesaba un nuevo bloque de muestras. Esto ocurría porque la plataforma ejecuta la función `work` varias veces de manera independiente, generando saltos o discontinuidades en la señal de salida. Para corregir este comportamiento, se agregó una variable que almacena el último valor acumulado, permitiendo mantener una integración continua entre un bloque de datos y el siguiente.

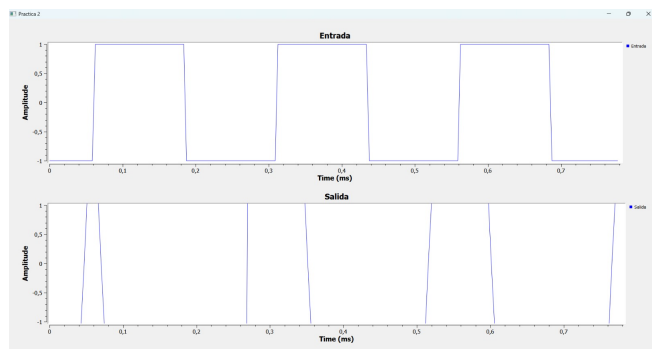


Figura 2: Simulación del funcionamiento del bloque acumulador.

III Conclusiones

Completar

Referencias

- [1] Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones, *Libro de Comunicaciones Plus Volumen I*. Bucaramanga, Colombia: Universidad Industrial de Santander, 2020, material de apoyo académico.